

Problem 178: Get Out the Vote

Coach's Resource Guide

Summary of Problem

Students must implement a ranked-choice voting system that is able to determine the victor from a series of ballots.

Topics Covered

- Ranked-choice voting
- Cybersecurity

Suggested Approach

As with many problems, we first need to begin with reading in a line indicating the amount of data we're dealing with. Upon reading in the line, it should be split at the space; the two resulting strings should then be converted into integers.

We should then prepare the other data structures we need. First, we'll need to declare an array or list of strings capable of holding each of the ballots. Each ballot will be represented as a string with a length equal to the number of candidates; due to how the system works, every ballot must include a vote for each candidate, even the voter's last choice. We'll also need a map to associate each candidate with the number of votes. This could also simply be an array of integers; since each candidate is identified by a letter, in alphabetical order, the first integer in that array would correspond to the votes for candidate A; the second integer shows the votes for B, and so on. Next, we'll need something to contain the list of eliminated candidates; this could be a list, but could also be an array or even a string, as we'll demonstrate later. Finally, we need to know how many rounds of counting we've done (starting at 0), and how many votes a candidate must receive in order to win; this is equal to the number of ballots, divided by two, then plus one. See the pseudocode below for everything we've created so far:

```
let countStrings = input.readLine().split(" ");
let ballotCount = countStrings[0] as int;
let candidateCount = countStrings[1] as int;
let ballots = new string[ballotCount];
let voteCounts = new int[candidateCount];
// or, let voteCounts = new int[candidateCount];
let eliminated = "";
let rounds = 0;
let targetCount = (ballotCount / 2) + 1;
let winner = null;
```

Reading in the ballots is next; each ballot should be added to the 'ballots' array or list, but shouldn't be processed just yet.

Actually counting the ballots will likely require multiple rounds of counting. We'll need to declare a loop, set to terminate once a winner is identified. Alternatively, you could declare this as an infinite loop (such as `while(true)`), but you must be sure to break out of the loop once a winner is identified. Within the loop, we'll want to first increment our 'rounds' counter, then count the number of votes for each candidate. Since each ballot (string) lists candidates (characters) in order of preference - favorite candidate first - we simply need to check the first character in each string to identify the desired vote. We then add one to the corresponding candidate's count, and move on to the next ballot.

```
while(winner == null){
    rounds++;

    for(let i = 0; i < ballotCount; i++){
        let vote = ballots[i][0]; // get first character
        let voteIndex = vote - 'A'; // characters are numbers too!
        voteCounts[voteIndex]++; // find and increment count
    }
    // ...
}
```

Once all of the votes are tallied, we check to see if there is a winner. A winner is declared once one of the candidates reaches the target vote count of 50%+1. If any candidate has reached this threshold, save their letter to the 'winner' variable so we can report who won. At the same time, we should also check the counts for which candidate had the *lowest* vote count. Some care has to be taken here; any candidate we've already eliminated will have a vote count of zero (you'll see why in a moment), but we also have to consider the possibility that a non-eliminated candidate could receive zero votes in the first few rounds. See the pseudocode below:

```
let lowestCount = ballotCount;
let loser = null;
for(let candidate = 'A'; candidate < 'A' + candidateCount; candidate++){
    if(voteCounts[candidate - 'A'] >= targetCount){
        winner = candidate;
    }
    if(voteCounts[candidate - 'A'] < lowestCount &&
        eliminated.indexOf(candidate) < 0){
        lowestCount = voteCounts[candidate - 'A'];
        loser = candidate;
    }
}
```

If we found a winner, we should break out of the loop here and stop processing ballots. Otherwise, we need to eliminate the candidate we identified as the loser in that round. We should add their letter to the 'eliminated' list/array/string, then remove that character from each of the ballots. Any ballots for which that candidate was the first choice will now have the second choice as the first character. In any ballots where that candidate was a second or subsequent choice, they'll be removed anyway, allowing us to move directly to the next choice in line one other preferences are eliminated. Finally, we should reset all of the vote counts to zero to allow us to prepare for the next round of tallies. See below:

```
if(winner == null){
    eliminated += loser;
    for(let i = 0; i < ballotCount; i++){
        ballots[i] = ballots[i].replace(loser, "");
    }
    for(let i = 0; i < candidateCount; i++){
        voteCounts[i] = 0;
    }
}
} // end of while(winner == null) loop
```

Once out of the while loop, we simply need to print out the results of the election. Calculate the percentage of votes the winner received by dividing their final vote count by the total number of ballots, and print the results.

Additional Background

Election security has been a hotly contested topic in the United States over the past few years. Subject to particular scrutiny is the use of electronic voting systems, which are becoming increasingly common as a way for governments to reduce the use of environmentally-unfriendly paper ballots and increase the speed of tallying results. Detractors of these systems claim that they threaten the security of elections because they can be easily hacked or otherwise manipulated into incorrectly skewing the results towards a particular candidate. In almost every case, however, these claims have been proven false.

Security is a major concern with many software applications, and as noted above, it is of particular concern with something that has major real-life impacts such as an election. But what is security, exactly? With any sort of computing system, security (or “cybersecurity”) is concerned primarily with three main points:

- Confidentiality - The knowledge that your data is kept secret and hidden from those not authorized to view it
- Integrity - The knowledge that your data has not been tampered with or altered in any way from its intended form
- Availability - The knowledge that you (and other authorized people) can access your data whenever it may be needed

Let’s look into some ways these concepts could be applied to an electronic voting system.

Confidentiality is a major point in a fair election. People casting their votes need to be confident that their votes are secret; that nobody knows specifically how they voted. Voting machines, as a result, often aren’t given any information about who is voting. If they do display information, it’s simply to confirm that the person voting is the person it expects to be voting; that data can then be discarded, and would not be associated with the actual vote. Most jurisdictions do rely on some sort of voter registration system to confirm that those voting are allowed to do so, but this is handled by human officials, not the voting machines, and again isn’t associated with the final ballot.

Another point in confidentiality is keeping the overall vote counts secure. While press agencies often conduct “exit polls” to get an idea of how the election is going as voting is taking place, reporters can’t interview everyone, and not everyone will respond (or choose to do so honestly). At the end of the day, the official count is what really matters. Unauthorized access to that data could skew an election, however. If one candidate were to become aware that they were in the lead halfway through the election, they could prematurely claim victory on the news and social media outlets. This may cause supporters of their opponent to think it’s not worth voting and skip casting their ballot... even if there would have been enough of those supporters to take the lead. So, how do we prevent these leaks from occurring?

Encrypting data is one of the best ways to ensure that data can only be read by those who should access it. Without the key to decrypt the data, it’s gibberish and therefore useless. As a result, any election related data can be encrypted by the machines, and will remain encrypted until it’s read by authorized election officials. Encryption can be broken, however, and so it can’t be the only means by which to rely on securing data. Fortunately, the simplest solutions are often some of the best. If there is no way to connect to the voting machines, there’s no way for anyone to access the data they hold. For this reason, voting machines are almost never connected to the internet or any other network. This means that the only way to access the data is by physically removing the memory chip that holds that data; which is itself likely locked up. Once the election is over, an election official can unlock the compartment, remove the data chip, and personally deliver it to a government office for counting. Anyone else attempting to access the chip would be very obvious and could be stopped before they were able to gain access to the chip, let alone the data it holds.

This lack of network access also supports our data integrity and availability. Even if a hacker can’t read encrypted data, they can still corrupt it or make it unusable by overwriting it with different information. Again, however, this would require direct physical access to the voting machine’s memory chip since it contains no network connections. Even so, election officials are often required to conduct audits of the elections they oversee, even if no foul play was suspected. This helps to confirm that counts are accurate and supports the “integrity” part of the security mission. To support this, voting machines can use a number of methods to confirm that their counts are accurate.

“Checksums” can be generated by hashing ballot data together; even a minor change to any part of any ballot will result in a wildly different checksum value, indicating that something is wrong. Blockchains, similar to those used for cryptocurrencies, can be used to reconstruct ballots in the order they were cast. Failing all else, a log simply stating information such as “Ballot 123456 was cast for candidate A” can provide a human-readable (and thus human-verifiable) record of every vote cast at that machine.

Another means by which to protect data integrity is simple validation; ensuring, even before the data is recorded, that it makes sense. In most election systems, a voter can pick only one candidate from those running. If two or more are selected, that’s invalid data. In the ranked choice voting system demonstrated in this problem, a voter must assign a preference to every candidate. If any candidate doesn’t have a preference, this is also invalid data. Even seemingly valid data may not be what the user

intended; if a voting machine locks in a choice as soon as a candidate is selected, an accidental touch could cause a voter to vote for a different candidate than they intended. Performing simple validation checks and adding confirmation mechanisms to prevent those situations are basic tenets of “secure coding,” the practice of ensuring that software does its part to support security.

These and other security methods are a critical component of ensuring fair and open elections in any democracy; and indeed, they are working. These same security principles can inform other aspects of our lives as well, particularly as we increasingly move into a digital world.

Further Discussion Topics

- What other methods could be used to secure an electronic voting system?
- Consider another system, such as the one used by your school to manage student grades. How could you apply the three tenets of cybersecurity - confidentiality, integrity, and availability - to that system?